

03-docker-realtime.md

ELI15 (your level, real explanation)

♦ What you did

1. Created a container:

```
docker run -it --name paytm ubuntu
```

2. Added files:

```
touch paytmfiles{1..5}
```

3. Created image:

```
docker commit paytm new-image-name
```

👉 This image now contains those files

♦ Then you created containers

```
docker run -it --name paytm-1 new-image-name
```

```
docker run -it --name paytm-2 new-image-name
```

👉 Both containers start with same data

Critical concept

Each container has its own:

Writable Layer

So:

- paytm-1 → changes stay in paytm-1
 - paytm-2 → completely separate
-

Why data doesn't sync

Because:

Container = isolated filesystem

✓ How to share data

Use volume:

```
docker run -it -v mydata:/data nginx
```

Now:

- All containers using `mydata` share same files

🔥 Real-world mapping (your NGINX + ports)

You ran:

```
docker run -d -p 9090:80 nginx
```

👉 This means:

Part	Meaning
9090	EC2 machine port
80	container port

👉 Access:

<http://EC2-IP:9090>

⚠ Important corrections (you had some confusion)

✗ Wrong understanding

EXPOSE or container IP is enough

👉 Not true

✓ You must use `-p` (port mapping)

✗ docker attach is safe

👉 No — risky

✓ Use:

```
docker exec -it cont-2 /bin/bash
```

Final answer to your main question

If I add files in paytm-1, will it reflect in paytm-2?

👉 ❌ NO (default behavior)

👉 ✅ Only with volumes

Final mental model

Image → template

Container → independent instance

Volume → shared storage

🧐 ELI5 (Explain Like I'm 5)

Imagine you are making a **cake** 🍰

- The **recipe** = Dockerfile
- The **cake** = Docker image
- Eating the cake = Running container

👉 Steps:

1. Write recipe (Dockerfile)
 2. Bake cake (build image)
 3. Serve cake (run container)
-

Ingredients (Dockerfile commands)

- **FROM** → what base cake you use
- **COPY** → add ingredients
- **RUN** → bake steps
- **CMD** → what happens when you serve

👉 Simple idea:

Dockerfile = instructions to make something ready to use

 **ELI10**

Think of Dockerfile like **installing an app on a new laptop**

- `FROM ubuntu` → install OS
- `COPY` → copy your app files
- `RUN` → install dependencies (like npm install)
- `CMD` → start your app

👉 Flow:

Dockerfile → Image → Container

- Image = saved setup
 - Container = running app
-

Important understanding

- `RUN` → setup time (install stuff)
 - `CMD` → runtime (start app)
-



ELI15 (your level, real understanding)

A Dockerfile defines **how to build a runtime environment**

Build stage

```
FROM node:18
WORKDIR /app
COPY . .
RUN npm install
```

👉 This creates the **image**

Run stage

```
CMD ["npm", "start"]
```

👉 This runs when container starts

Key concepts

- Image = snapshot of environment
 - Container = running instance of that snapshot
-

Important differences

Instruction	Purpose
RUN	executes during build
CMD	runs when container starts
ENTRYPOINT	fixed startup command
ENV	runtime variables
ARG	build-time variables

Real-world mapping (your system)

You already do:

- Blue/Green apps
- NGINX switching

Docker fits like:

Docker Image → your app version

Container → running blue / green

NGINX → routes traffic

Final simple sentence (remember this)

👉 Dockerfile = blueprint

👉 Image = built package

👉 Container = running app